Experiment No. : 2

Title: Understanding Data with Programming Exercise

Batch:A1 **Roll No.:16014225023** **Experiment No.:2**

Aim: Understanding of the Data Handling technique with Programming exercise

---

Resources needed: Resources needed: Python 3.x environment (Jupyter Notebook / Google Colab etc.),numpy and pandas library

---

Theory

There are several programming languages that are commonly used for data handling, analysis, and manipulation tasks. Some of the most popular data handling programming languages are Python, R, Scala, Matlab, Julia, and SQL. In this practical session, we will handle Python-based data handling commands. Let us see features of Python.

Python

Python is one of the most widely used programming languages for data handling and analysis. It has rich libraries like Pandas, NumPy, Matplotlib, Seaborn, and SciPy that offer comprehensive support for data manipulation, numerical computing, and data visualization. Python's readability and user-friendly syntax make it a top choice for data scientists, researchers, and analysts. In this session, we will learn some important Python libraries, namely NumPy and Pandas.

NumPy

NumPy, which stands for "Numerical Python," is a fundamental package for numerical computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with an extensive collection of mathematical functions to operate on these arrays efficiently. NumPy is the backbone of many other Python libraries for scientific computing and data analysis.

Key features of NumPy:

- **N-dimensional array object (numpy.ndarray):** Efficient data structure for storing and manipulating large datasets, making it suitable for numerical computations and data analysis.
- **Broadcasting:** NumPy automatically broadcasts arrays with different shapes for element-wise operations, reducing the need for explicit loops.
- **Mathematical Functions:** NumPy offers a wide range of mathematical functions like trigonometric, exponential, statistical, and linear algebra functions, among others.
- **Indexing and Slicing:** NumPy supports advanced indexing techniques to access specific elements or subsets of data quickly.
- **Integration with C/C++:** NumPy is written in C and Python, which provides the advantage of speed and efficiency in numerical computations.

Pandas

Pandas is an open-source data manipulation library built on top of NumPy. It provides high-level data structures and functions to make working with structured data easy and intuitive. The two primary data structures in Pandas are the **Series** and **DataFrame**.

Key features of Pandas:

- **Series:** A one-dimensional labeled array, similar to a NumPy array but with additional indexing capabilities, making it more versatile for data analysis.

- **DataFrame:** A two-dimensional labeled data structure with rows and columns, resembling a spreadsheet or SQL table, which is the central data structure for data analysis in Pandas.
- **Data Import/Export:** Pandas can read and write data from various file formats like CSV, Excel, SQL databases, JSON, and HTML tables.
- **Data Exploration:** Pandas provides functions to explore, clean, and preprocess data, such as handling missing values, filtering, grouping, and aggregation.
- **Time-Series Analysis:** Pandas has robust support for handling time-series data, making it popular for financial and temporal data analysis.
- **Data Visualization:** Although not a visualization library, Pandas integrates well with other visualization libraries like Matplotlib and Seaborn, allowing easy creation of basic visualizations directly from DataFrames and Series.

Together, NumPy and Pandas form a powerful combination for data analysis and data manipulation tasks in Python. NumPy handles the underlying numerical computations and efficient data storage, while Pandas provides high-level structures and functions that simplify data handling and analysis tasks.

Exercise: Data Handling Commands with Python

Both Pandas and NumPy offer a wide range of commands and functions for data analysis. Here are some essential commands from each library commonly used in data analysis:

1. Pandas

Creating DataFrames

- pd.DataFrame(data, columns=['col1', 'col2', ...]): Creates a DataFrame from data (a dictionary, list of lists, or a NumPy array) with specified column names.

Data Exploration

- df.head(n): Returns the first n rows of the DataFrame.
- df.info(): Provides information about the DataFrame, including data types and non-null counts.
- df.describe(): Generates summary statistics for numerical columns.
- df.shape: Returns the dimensions of the DataFrame (rows, columns).
- df.columns: Returns the column names.

Data Selection and Filtering

- df['column_name']: Selects a specific column from the DataFrame.
- df.loc[row_index, 'column_name']: Accesses specific data using labels for both rows and columns.
- df.iloc[row_index, col_index]: Accesses specific data using integer-based indexing.
- df[df['column_name'] > value]: Filters data based on a condition.

Data Aggregation

- df.groupby('column_name').agg({'other_column': 'mean'}): Grouping and aggregation based on specific columns.
- df['column_name'].value_counts(): Calculates the frequency of unique values in a column.

Data Cleaning

- df.isnull(): Checks for missing values in the DataFrame.
- df.dropna(): Drops rows with missing values.
- df.fillna(value): Fills missing values with a specific value.

Data Visualization

- df['column_name'].plot(kind='hist'): Creates a histogram for a specific column.

- df.plot(x='x_column', y='y_column', kind='scatter'): Creates a scatter plot.

2. NumPy

Creating Arrays

- np.array([1, 2, 3]): Creates a 1-dimensional NumPy array.
- np.zeros((m, n)): Creates an $m \times n$ array filled with zeros.
- np.ones((m, n)): Creates an $m \times n$ array filled with ones.
- np.arange(start, stop, step): Creates an array with values from start to stop (exclusive) with a given step size.

Array Operations

- np.mean(arr): Calculates the mean of the array.
- np.median(arr): Calculates the median of the array.
- np.sum(arr): Calculates the sum of the array elements.
- np.std(arr): Calculates the standard deviation of the array.
- np.unique(arr): Returns the unique elements of the array.

Array Indexing and Slicing

- arr[index]: Accesses specific elements of the array using indexing.
- arr[start:stop]: Slices the array from start to stop (exclusive).

Array Broadcasting

- NumPy automatically broadcasts arrays with different shapes for element-wise operations.

Mathematical Functions

- np.sin(arr), np.cos(arr), np.exp(arr), etc.: Various mathematical functions that operate element-wise on the array

---

Activity:

- Read dataset related to your chosen AI use case using Numpy and Pandas
- Perform in detail data analysis of this dataset
- Get information and description of dataset.
- See if any null values are present. Display count of null values.
- Choose appropriate technique to handle missing values.(imputation with use of inplace)
- Use sorting of data in dataframe to display topmost 5 or 8 records based on one or more column values(conditional filtering)
- Get frequency listing of any one relavant column(2 cases)
- Sorting of rows and columns,(implicit and explicit indexing)
- Accessing particular row based on certain condition and displaying only one or few columns from it.(3 cases with compound conditions)
- Minimum and maximum values related analysis
- Use of group by on one or more columns(2 cases)
- Adding new column to existing dataframe and polulating same using existing columns data.
- Use of appropriate aggregate functions with groupby.(2 cases)

- Selection on particular groups based on name or condition
- Find correlation between any two columns values.
- Try transformation(normalization using any technique) on data set
- Joining, merging and concatenation of data in dataframe.

Write down observation for your dataset for each of above listed task of analysis.

Results: (python script and output and inferences derived)

```
import numpy as np

import pandas as pd



data = {

      'Customer_ID': ['1001', '1002', '1003', '1004', '1005', '1006',
'1007', '1008', '1009', '1010', '1011', '1012', '1013', '1014', '1015'],

      'Gender': ['Male', 'Female', 'Female', 'Male', 'Male', 'Female',
'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female',
'Male'],

    'Age': [28, 45, 63, 34, 52, 23, 41, 59, 31, 48, 36, 60, 25, 50, 42],

    'Tenure_Months': [12, 48, 3, 24, 60, 1, 36, 72, 8, 54, 18, 65, 5, 42,
30],

    'Contract_Type': ['Monthly', 'Two Year', 'Monthly', 'One Year', 'Two
Year', 'Monthly', 'One Year', 'Two Year', 'Monthly', 'Two Year', 'One
Year', 'Two Year', 'Monthly', 'One Year', 'Monthly'],

     'Payment_Method': ['Electronic Check', 'Bank Transfer', 'Electronic
Check', 'Credit Card', 'Bank Transfer', 'Electronic Check', 'Credit
Card', 'Bank Transfer', 'Electronic Check', 'Credit Card', 'Bank
Transfer', 'Credit Card', 'Electronic Check', 'Bank Transfer',
'Electronic Check'],

      'Monthly_Charges': [65.5, 89.0, np.nan, 75.2, 105.0, 45.0, 80.0,
115.5, np.nan, 95.0, 70.0, 110.0, 50.0, 85.0, 68.0],
```

```
    'Total_Charges': [786.0, 4272.0, 150.0, 1804.8, 6300.0, 45.0, 2880.0,
8316.0, 440.0, np.nan, 1260.0, 7150.0, 250.0, 3570.0, 2040.0],

    'Churn': ['No', 'No', 'Yes', 'No', 'No', 'Yes', 'No', 'No', 'Yes',
'No', 'No', 'No', 'Yes', 'No', 'Yes']

}



df = pd.DataFrame(data)

display(df.head())
```

	Customer_ID	Gender	Age	Tenure_Months	Contract_Type	Payment_Method	Monthly_Charges	Total_Charges	Churn
0	1001	Male	28	12	Monthly	Electronic Check	65.5	786.0	No
1	1002	Female	45	48	Two Year	Bank Transfer	89.0	4272.0	No
2	1003	Female	63	3	Monthly	Electronic Check	NaN	150.0	Yes
3	1004	Male	34	24	One Year	Credit Card	75.2	1804.8	No
4	1005	Male	52	60	Two Year	Bank Transfer	105.0	6300.0	No

```
print("Dataset Information:")

df.info()



print("\nSummary Statistics:")

display(df.describe())



print("\nNull Values Count:")

print(df.isnull().sum())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Customer_ID      15 non-null     object
 1   Gender           15 non-null     object
 2   Age              15 non-null     int64
 3   Tenure_Months    15 non-null     int64
 4   Contract_Type    15 non-null     object
 5   Payment_Method   15 non-null     object
 6   Monthly_Charges  13 non-null     float64
 7   Total_Charges    14 non-null     float64
 8   Churn            15 non-null     object
dtypes: float64(2), int64(2), object(5)
memory usage: 1.2+ KB

Summary Statistics:
```

	Age	Tenure_Months	Monthly_Charges	Total_Charges
count	15.000000	15.000000	13.000000	14.000000
mean	42.466667	31.866667	81.015385	2804.557143
std	12.894443	24.053116	21.804313	2760.225650
min	23.000000	1.000000	45.000000	45.000000
25%	32.500000	10.000000	68.000000	526.500000
50%	42.000000	30.000000	80.000000	1922.400000
75%	51.000000	51.000000	95.000000	4096.500000
max	63.000000	72.000000	115.500000	8316.000000

```
Null Values Count:
Customer_ID        0
Gender             0
Age                0
Tenure_Months      0
Contract_Type      0
Payment_Method     0
Monthly_Charges    2
Total_Charges      1
Churn              0
dtype: int64
```

```
mean_monthly = df['Monthly_Charges'].mean()

df.fillna({'Monthly_Charges': mean_monthly}, inplace=True)
```

```
median_total = df['Total_Charges'].median()

df.fillna({'Total_Charges': median_total}, inplace=True)



print("Null counts after imputation:")

print(df.isnull().sum())
```

K. J. SOMAIYA COLLEGE OF ENGG.

```
Null counts after imputation:
Customer_ID        0
Gender             0
Age                0
Tenure_Months      0
Contract_Type      0
Payment_Method     0
Monthly_Charges    0
Total_Charges      0
Churn              0
dtype: int64
```

```
filtered_df = df[df['Monthly_Charges'] > 60]

sorted_top_records      =      filtered_df.sort_values(by='Total_Charges',
ascending=False)

print("Top 8 Records (Monthly Charges > 60, sorted by Total Charges):")

display(sorted_top_records.head(8))



print("\nFrequency of Contract Type:")

print(df['Contract_Type'].value_counts())
```

```
print("\nFrequency of Payment Method:")

print(df['Payment_Method'].value_counts())



print("\nImplicit Indexing:")

display(df.iloc[0:3, 0:4])



print("\nExplicit Indexing:")

display(df.loc[0:2, ['Customer_ID', 'Age', 'Contract_Type']])



print("\nCompound Condition 1:")

cond1 = df[(df['Age'] > 40) & (df['Churn'] == 'Yes')]

display(cond1[['Customer_ID', 'Age', 'Monthly_Charges', 'Churn']])



print("\nCompound Condition 2:")

cond2 = df[(df['Tenure_Months'] >= 24) & (df['Contract_Type'] == 'Two
Year')]

display(cond2[['Customer_ID', 'Tenure_Months', 'Contract_Type']])



print("\nCompound Condition 3:")

cond3 = df[(df['Monthly_Charges'] > 80) | (df['Gender'] == 'Female')]

display(cond3[['Customer_ID', 'Gender', 'Monthly_Charges']].head())
```

Top 8 Records (Monthly Charges > 60, sorted by Total Charges):

	Customer_ID	Gender	Age	Tenure_Months	Contract_Type	Payment_Method	Monthly_Charges	Total_Charges	Churn
7	1008	Female	59	72	Two Year	Bank Transfer	115.5	8316.0	No
11	1012	Female	60	65	Two Year	Credit Card	110.0	7150.0	No
4	1005	Male	52	60	Two Year	Bank Transfer	105.0	6300.0	No
1	1002	Female	45	48	Two Year	Bank Transfer	89.0	4272.0	No
13	1014	Female	50	42	One Year	Bank Transfer	85.0	3570.0	No
6	1007	Male	41	36	One Year	Credit Card	80.0	2880.0	No
14	1015	Male	42	30	Monthly	Electronic Check	68.0	2040.0	Yes
9	1010	Female	48	54	Two Year	Credit Card	95.0	1922.4	No

Frequency of Contract Type:

```
Contract_Type
Monthly     6
Two Year    5
One Year    4
Name: count, dtype: int64
```

Frequency of Payment Method:

```
Payment_Method
Electronic Check    6
Bank Transfer       5
Credit Card         4
Name: count, dtype: int64
```

Implicit Indexing:

	Customer_ID	Gender	Age	Tenure_Months
0	1001	Male	28	12
1	1002	Female	45	48
2	1003	Female	63	3

Explicit Indexing:

	Customer_ID	Age	Contract_Type
0	1001	28	Monthly
1	1002	45	Two Year
2	1003	63	Monthly

Compound Condition 1:

	Customer_ID	Age	Monthly_Charges	Churn
2	1003	63	81.015385	Yes
14	1015	42	68.000000	Yes

Compound Condition 2:

	Customer_ID	Tenure_Months	Contract_Type
1	1002	48	Two Year
4	1005	60	Two Year
7	1008	72	Two Year
9	1010	54	Two Year
11	1012	65	Two Year

Compound Condition 3:

	Customer_ID	Gender	Monthly_Charges
1	1002	Female	89.000000
2	1003	Female	81.015385
4	1005	Male	105.000000
5	1006	Female	45.000000
7	1008	Female	115.500000

```
max_charge = df['Monthly_Charges'].max()

min_charge = df['Monthly_Charges'].min()

print("Maximum Monthly Charge:", max_charge)

print("Minimum Monthly Charge:", min_charge)



print("\nAggregations by Contract Type:")

grouped_contract = df.groupby('Contract_Type')

display(grouped_contract.agg({'Monthly_Charges': ['mean', 'min', 'max'],
'Tenure_Months': 'mean'}))
```

```
print("\nAggregations by Churn:")

grouped_churn = df.groupby('Churn')

display(grouped_churn.agg({'Total_Charges':   ['sum',   'mean'],   'Age':
'median'}))



df['Annual_Estimated_Cost'] = df['Monthly_Charges'] * 12



print("\nSelection of Two Year Group:")

two_year_group = grouped_contract.get_group('Two Year')

display(two_year_group[['Customer_ID',                'Monthly_Charges',
'Total_Charges']])



correlation = df['Monthly_Charges'].corr(df['Total_Charges'])

print("\nCorrelation between Monthly and Total Charges:", correlation)



min_val = df['Monthly_Charges'].min()

max_val = df['Monthly_Charges'].max()

df['Monthly_Charges_Norm'] = (df['Monthly_Charges'] - min_val) / (max_val
- min_val)



df_services = pd.DataFrame({

      'Customer_ID': ['1001', '1002', '1003', '1004', '1005', '1006',
'1007', '1008', '1009', '1010', '1011', '1012', '1013', '1014', '1015'],
```

```
    'Internet_Service': ['Fiber', 'DSL', 'DSL', 'Fiber', 'Fiber', 'DSL',
'Fiber', 'Fiber', 'DSL', 'Fiber', 'DSL', 'Fiber', 'DSL', 'Fiber', 'DSL']

})



merged_df = pd.merge(df, df_services, on='Customer_ID')

print("\nMerged Sample:")

display(merged_df[['Customer_ID',                          'Contract_Type',
'Internet_Service']].head())
```

Aggregations by Contract Type:

Contract_Type	Monthly_Charges			Tenure_Months
	mean	min	max	mean
Monthly	65.088462	45.0	81.015385	9.833333
One Year	77.550000	70.0	85.000000	30.000000
Two Year	102.900000	89.0	115.500000	59.800000

Aggregations by Churn:

Churn	Total_Charges		Age
	sum	mean	median
No	38261.2	3826.12	46.5
Yes	2925.0	585.00	31.0

Selection of Two Year Group:

	Customer_ID	Monthly_Charges	Total_Charges
1	1002	89.0	4272.0
4	1005	105.0	6300.0
7	1008	115.5	8316.0
9	1010	95.0	1922.4
11	1012	110.0	7150.0

Correlation between Monthly and Total Charges: 0.8548056922621441

Merged Sample:

	Customer_ID	Contract_Type	Internet_Service
0	1001	Monthly	Fiber
1	1002	Two Year	DSL
2	1003	Monthly	DSL
3	1004	One Year	Fiber
4	1005	Two Year	Fiber

Post-Lab Questions:

1. **Explain the concept of "Broadcasting" in NumPy with the help of suitable example**
   Broadcasting is a fundamental mechanism in NumPy that automatically allows arrays with different shapes to be used together in element-wise operations, significantly reducing the need to write explicit loops. Instead of throwing an error when shapes don't match, NumPy virtually "stretches" or broadcasts the smaller array across the larger one to make their dimensions compatible.
   Example: Adding a 1D Array to a 2D Array If you have a 2D array representing student marks in various subjects and you want to add specific grace marks to each subject, you can add a 1D array directly to the 2D array without writing a `for` loop.
   Code:
   import numpy as np
   Shape is (3, 3)
   marks = np.array([[70, 80, 90], [65, 75, 85], [50, 60, 70]])
   grace_marks = np.array([5, 2, 0])
   updated_marks = marks + grace_marks print(updated_marks)
   Output:
   [[75 82 90]
   [70 77 85]
   [55 62 70]]

J. SOMAIYA COLLEGE OF ENGG.

2. **Explain the difference between label-based indexing (df.loc[]) and integer-based indexing (df.iloc[]) in Pandas with the help of suitable example.**

Label-based indexing (df.loc[]) accesses specific data using the explicitly defined labels for both rows and columns. When you slice a DataFrame using .loc[], the slice is **inclusive** of the final label.

Integer-based indexing (df.iloc[]) accesses specific data using the implicit, zero-based integer positions of the rows and columns, functioning exactly like standard Python lists. When you slice a DataFrame using .iloc[], the final index is **exclusive**.

Example: Consider a DataFrame where the rows have custom string labels instead of default numbers.

```
None
import pandas as pd

# Creating a DataFrame with custom row labels
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
        'Age': [25, 30, 35, 40]}
df = pd.DataFrame(data, index=['row1', 'row2', 'row3',
'row4'])
```

Using df.loc[]: If you want to extract the names of the first three people, you must use their explicit row and column labels. Notice that 'row3' is included in the output.

```
None
print(df.loc['row1':'row3', 'Name'])
```

Output:

```
None
row1      Alice
row2        Bob
row3    Charlie
Name: Name, dtype: object
```

Using df.iloc[]: If you want to extract the exact same data using integer positions, you target rows 0 through 2 and column 0 (the 'Name' column). Because .iloc[] is exclusive of the endpoint, you must slice up to index 3.

```
None
print(df.iloc[0:3, 0])
```

Output:

```
None
row1      Alice
row2        Bob
row3    Charlie
Name: Name, dtype: object
```

Course Outcomes: (refer exp list)

CO1: Implement foundational Python libraries (NumPy and Pandas) to load, explore, and summarize structured datasets.

CO2: Apply data cleaning and preprocessing techniques, including in-place missing value imputation and Min-Max normalization, to prepare raw data for downstream analysis.

CO3: Formulate complex data retrieval queries using implicit (`.iloc`) and explicit (`.loc`) indexing, alongside compound conditional filtering.

CO4: Execute advanced data manipulation tasks such as multidimensional `groupby` aggregations, statistical correlation, and DataFrame merging to extract analytical insights.

Conclusion:

In this experiment, essential data handling and exploratory data analysis (EDA) techniques were successfully implemented using Python's Pandas and NumPy libraries on a Customer Churn dataset. Raw structured data was loaded into a DataFrame and comprehensively analyzed to understand its structural properties, distribution, and completeness. Missing values in continuous features (Monthly_Charges and Total_Charges) were effectively resolved using central tendency imputation (mean and median) directly within the dataset.

The practical application of conditional filtering and logical indexing (.loc and .iloc) demonstrated how to accurately isolate specific data subsets, such as identifying high-value, long-tenure customers on two-year contracts. Analytical operations, including grouping data to find segmented aggregate bounds and calculating the positive correlation between monthly and total charges ($r \approx 0.85$), yielded clear quantitative relationships. Finally, executing mathematical transformations like Min-Max normalization and successfully merging a secondary internet services DataFrame reinforced the fundamental data preprocessing and integration skills necessary for building robust artificial intelligence and data science pipelines.

K. J. SOMAIYA COLLEGE OF ENGG.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

Books/ Journals/ Websites:

1. S.C. Gupta, V. K. Kapoor, *Fundamentals of Mathematical Statistics*, Sultan Chand and Sons, 2020.
2. P. N. Tan, M. Steinbach, Vipin Kumar, *Introduction to Data Mining*, Pearson Education, 2016.
3. Han, Kamber, *Data Mining Concepts and Techniques*, Morgan Kaufmann, 3rd Edition, 2012
4. NumPy Documentation - https://numpy.org/doc/
5. Pandas Documentation - https://pandas.pydata.org/